Implementable Compressed-Block Storage for Full-State Quantum Circuit Simulation

Daniel Adrián González Buendía, Pablo Josue Rojas Yepes, and Gilberto Javier Díaz Toro

Universidad Industrial de Santander, Bucaramanga, Colombia

Abstract. Full-state quantum circuit simulation remains the reference method for exact execution because every amplitude of the n-qubit register is represented explicitly. Its limiting resource is memory: the state vector contains 2^n complex amplitudes, so the storage requirement grows exponentially with the number of qubits. Compression is attractive in this setting, but it is useful for live simulation only when the compressed representation is the primary storage format during execution rather than an external persistence mechanism.

This paper presents an implementable compressed-memory architecture for full-state simulators. The design keeps the simulator interface at the level of a logical dense state vector while storing the physical state as independent compressed blocks, a bounded cache of dense materializations, and metadata that controls block authority, writeback, and recovery. The paper specifies the state partition, block descriptor, cache state machine, synchronization rules, and block-acquisition protocol required to execute gates over compressed storage. It also derives the block-access conditions induced by gate stride and shows how single-qubit and controlled operations can be routed through intra-block and inter-block execution plans. The result is a storage architecture that a simulator developer can implement directly: it identifies which structures must exist, which transitions are legal, when recompression occurs, and how logical state semantics are preserved while dense residency remains bounded.

Keywords: Quantum circuit simulation · Full-state simulation · Compressed memory · State vector · Memory management

1 Introduction

Exact state-vector simulation is still the most direct way to validate quantum circuits, verify compiler transformations, inspect intermediate amplitudes, and compare software and hardware executions. In this model, an n-qubit register is represented as

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle, \tag{1}$$

with one complex amplitude for each computational basis state [9]. A double-precision implementation therefore needs

$$M_{\text{dense}} = 16 \cdot 2^n \text{ bytes} \tag{2}$$

just for the state array. This storage cost dominates practical full-state simulation even in optimized simulators such as QuEST and ProjectQ [5, 10, 4, 2].

Compression-based simulation addresses the same bottleneck from a different angle: instead of replacing the state-vector model, it attempts to store amplitude data more compactly [14, 13, 15]. That direction is useful only if compressed storage participates in the execution path. If gate kernels still require a permanently dense global array, then compression reduces checkpoint or output size, but not the dense working set that constrains simulation.

The central systems problem is therefore not merely how to compress amplitudes, but how to execute state-vector simulation while the primary physical representation is compressed. An implementation needs clear answers to the following questions.

- How is the logical state mapped to compressed storage units?
- When a gate needs amplitudes, which blocks are materialized and for how long?
- What metadata determines whether the compressed payload or the cached dense copy is authoritative?
- When can a block be evicted, and when must it be recompressed first?
- How does the backend expose a dense programming model to the simulator while managing a compressed physical layout internally?

This paper answers those questions with a block-based storage architecture meant to be implemented directly. The contribution is the storage contract itself: the backend structures, invariants, block transitions, and gate-access protocol that make compressed execution practical while preserving the semantics of a dense state vector.

The design is guided by four principles.

1. The simulator continues to address a logical dense vector.
2. The backend owns compression, decompression, cache management, and write-back.
3. Dense memory is reserved for the current working set rather than for the whole register.
4. Every legal backend transition preserves one authoritative version of each logical block.

The rest of the paper develops the architecture from the storage model upward: first the logical-to-physical mapping, then the block metadata and cache state machine, then the gate planning rules needed to drive execution through those structures.

2 Execution Model and Backend Contract

A compressed backend must preserve the exact same logical state semantics as a conventional dense backend. The simulator therefore interacts with a *logical state vector*, while the backend maintains a *physical state* optimized for bounded residency.

2.1 Logical State

The logical state is indexed exactly as in a dense simulator. Basis-state numbering, endianness conventions, gate semantics, amplitude reads, and state export all operate over the same logical ordering used by a conventional state vector. From the simulator's perspective, an access to amplitudes α_i and α_j is still an access to those logical positions, regardless of how the backend stores them internally.

2.2 Physical State

The physical state is the implementation-owned representation. It contains:

- a partition of the logical vector into contiguous blocks of amplitudes,
- one compressed payload per block,
- a bounded cache for dense materializations of recently used blocks, and
- metadata for block mapping, cache placement, authority, and writeback.

Figure 1 summarizes this separation. The simulator emits operations over logical amplitudes; the backend translates those operations into block acquisition, dense execution, dirty marking, eviction, and recompression.

2.3 Required Backend Operations

An implementation only needs a compact set of operations, but those operations must be precise.

Table 1. Backend operations required to execute over compressed blocks.

Operation	Responsibility
Initialize state	Create block payloads for basis, zero, or externally supplied states.
Plan access	Determine which logical blocks a gate instance needs.
Acquire block(s)	Ensure that required blocks are resident in dense form and return mutable dense views.
Apply kernel	Execute the dense numerical kernel over the acquired block views.
Mark dirty	Record that a materialized block now contains newer data than its compressed payload.
Flush block(s)	Recompress dirty materializations and update the compressed store.
Export state	Flush pending dirty blocks and emit amplitudes in canonical logical order.

These operations define the backend contract. Everything else is an implementation detail built on top of them.

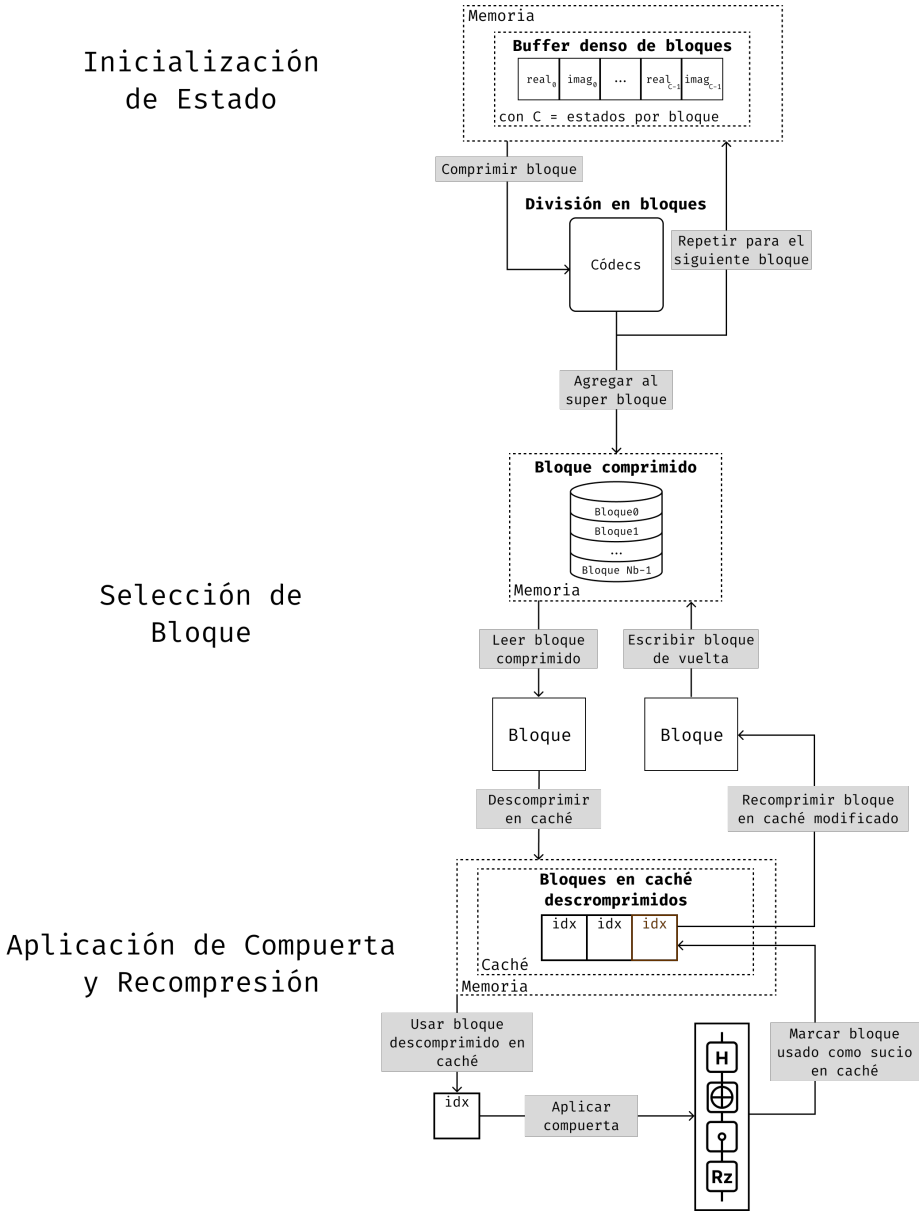

Fig. 1. Compressed-block backend for a full-state simulator. Logical state access remains unchanged at the simulator level; the backend resolves each access through block mapping, dense materialization, and compressed persistence.

3 State Partition and Block Metadata

3.1 Partitioning the Logical Vector

Let B be the number of amplitudes per block. The logical vector is divided into contiguous regions of size B, with the last region shorter if 2^n is not a multiple of B. Logical amplitude α_i maps to block

$$j(i) = \left\lfloor \frac{i}{B} \right\rfloor \tag{3}$$

and local offset

$$o(i) = i \bmod B. \tag{4}$$

The pair $(j(i), o(i))$ is the only mapping needed to route a logical index into the storage layer. Figure 2 shows the layout.

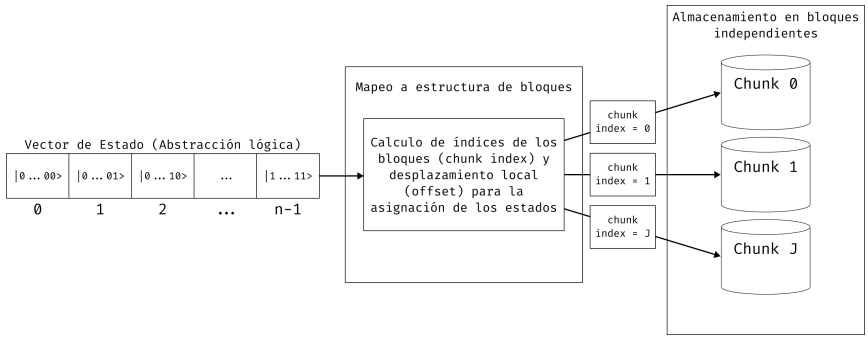

Fig. 2. Contiguous partition of the logical state vector into independently stored blocks. Each logical amplitude index is routed to one block identifier and one local offset.

The block size is part of the execution model, not just a storage parameter. It controls three effects simultaneously: the granularity of compression, the amount of data recovered on a cache miss, and the probability that a gate can execute entirely within one materialized block.

3.2 Block Descriptor

Each block is represented by a descriptor held in backend metadata. A practical implementation can store the descriptor as a struct or record with the fields in Table 2.

The descriptor belongs to the backend, not to the simulator. Gate kernels should never observe or modify these fields directly.

Table 2. Minimal metadata stored for each logical block.

Field	Type	Purpose
block_id	integer	Logical block identifier j.
logical_size	integer	Number of valid amplitudes in the block.
offset_bytes	integer	Location of the compressed payload in the backing store.
payload_bytes	integer	Compressed payload length.
cache_slot	integer / null	Dense cache slot if materialized.
state	enum	One of `COMPRESSED`, `CACHED_CLEAN`, `CACHED_DIRTY`.
last_use	integer / timestamp	Replacement-policy information.
codec_tag	integer / enum	Codec or preprocessing configuration used for this payload.

3.3 Authoritative Representation

The compressed backend relies on one strict ownership rule.

Authority rule. Every logical block has exactly one authoritative representation at any time:

- when the block state is `COMPRESSED` or `CACHED_CLEAN`, the compressed payload is authoritative;
- when the block state is `CACHED_DIRTY`, the dense cache slot is authoritative.

This rule eliminates ambiguity during recovery and writeback. The backend never needs to reconcile two competing versions of the same logical block.

4 Cache State Machine and Legal Transitions

The dense cache stores only currently useful block materializations. The cache may be implemented as a fixed-size array of block buffers plus policy metadata, for example an LRU queue. Figure 3 illustrates the intended behavior.

4.1 States

Each block descriptor occupies exactly one of three states.

- `COMPRESSED`: no dense copy exists in cache; the payload stored in compressed form is current.
- `CACHED_CLEAN`: a dense copy exists in cache and matches the compressed payload.
- `CACHED_DIRTY`: a dense copy exists in cache and is newer than the compressed payload.

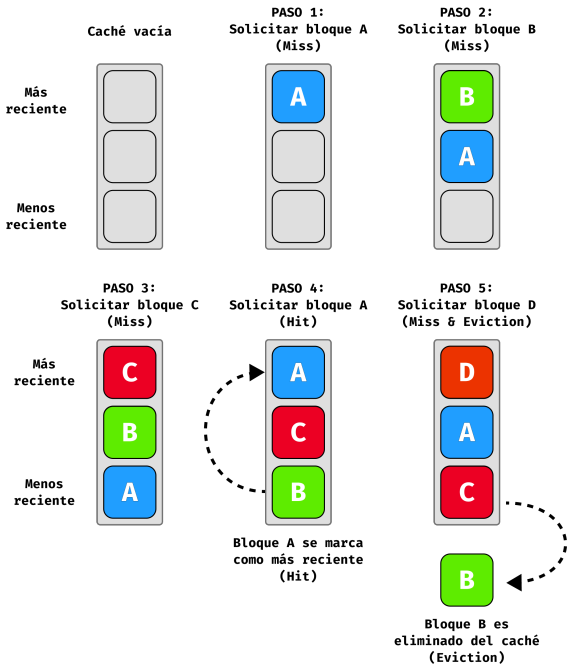

Fig. 3. Bounded dense cache for materialized blocks. Reuse is explicit; eviction is allowed only for clean blocks, while dirty blocks must be flushed before their slots can be reclaimed.

4.2 Transitions

The legal state transitions are fixed.

1. **Load:** `COMPRESSED` → `CACHED_CLEAN`. The backend allocates a cache slot and reconstructs the dense block from its compressed payload.
2. **Mutate:** `CACHED_CLEAN` → `CACHED_DIRTY`. A gate kernel modifies at least one amplitude in the materialized block.
3. **Flush:** `CACHED_DIRTY` → `CACHED_CLEAN`. The backend recompresses the dense block and overwrites the corresponding compressed payload.
4. **Evict:** `CACHED_CLEAN` → `COMPRESSED`. The dense slot is released while the payload remains current in compressed storage.

No direct transition from `CACHED_DIRTY` to `COMPRESSED` is legal. A dirty block must be flushed before its slot can be reused.

4.3 Backend Invariants

The implementation should enforce four invariants continuously.

Index invariant.
Every logical amplitude index maps to exactly one block identifier and one local offset.

Authority invariant.
Each block has exactly one authoritative representation, determined by its state.

Cache-slot invariant.
A cache slot contains at most one block, and a materialized block occupies exactly one cache slot.

Flush invariant.
After a global flush, every block is either `COMPRESSED` or `CACHED_CLEAN`, and compressed storage matches the full logical state.

These invariants are sufficient to preserve logical correctness throughout decompression, mutation, writeback, and eviction.

5 Block Acquisition, Eviction, and Writeback

A compressed backend is implementable only if block acquisition is deterministic. The procedure below is the core of the design.

5.1 AcquireBlock(j)

For a logical block identifier j, the backend executes the following steps.

1. Read descriptor D_j.
2. If D_j.state $\in \{\texttt{CACHED_CLEAN}, \texttt{CACHED_DIRTY}\}$, refresh replacement metadata and return the address of its dense cache slot.

3. Otherwise select a free cache slot; if no free slot exists, select a victim according to the replacement policy.
4. If the victim block is dirty, flush it: compress the dense slot, overwrite the victim payload in the backing store, and update the victim descriptor to `CACHED_CLEAN`.
5. Evict the clean victim by clearing its cache-slot mapping and changing its state to `COMPRESSED`.
6. Decompress block j into the selected slot, store the slot identifier in D_j, set D_j.state = `CACHED_CLEAN`, refresh replacement metadata, and return the slot.

This procedure is the only legal way to obtain a dense view of a compressed block.

5.2 Dirty Marking

After a dense kernel writes to a block, the backend sets the corresponding descriptor to `CACHED_DIRTY`. No recompression is forced immediately. Delayed write-back is essential because consecutive gates often reuse the same working set, and recompressing after every update would destroy locality.

5.3 FlushDirty(S)

Given a set of materialized dirty blocks S, the backend flushes each block using the same steps.

1. Read the dense data from its cache slot.
2. Apply any reversible preprocessing required by the selected codec.
3. Compress the block.
4. Write the payload to the backing store and update offset and payload length metadata if needed.
5. Change the descriptor state from `CACHED_DIRTY` to `CACHED_CLEAN`.

A full synchronization flush simply applies `FlushDirty` to all dirty blocks currently resident in cache.

6 Gate Planning Over the Block Layout

The simulator still emits logical gates, but the backend must convert each gate invocation into a block-access plan before the dense kernel runs.

6.1 Single-Qubit Pair Structure

A single-qubit gate acting on target qubit q updates amplitude pairs $(i, i \oplus 2^q)$ for all indices i whose q-th bit is zero [9]. The stride of the pair relation is therefore

$$s(q) = 2^q. \tag{5}$$

For a block size $B = 2^b$, the target qubit immediately determines whether pair partners remain inside one block.

Proposition 1. If $q < b$, every pair $(i, i \oplus 2^q)$ remains within a single block. If $q \geq b$, cross-block pairs occur.

Reason. A block boundary appears every 2^b consecutive logical indices. Flipping bit $q < b$ changes only lower-order bits inside the boundary interval, while flipping bit $q \geq b$ changes a bit that contributes to the block number.

This proposition is operational: it tells the backend whether one block view is enough or whether several blocks must be acquired together.

6.2 Execution Plans

The backend benefits from classifying gate invocations into execution plans.

Intra-block plan. Used when every touched pair lies inside one block. The backend iterates block by block:

1. acquire one block,
2. apply the dense kernel to the local offsets involved in that block,
3. mark the block dirty,
4. continue with the next logical block.

This is the cheapest path because only one materialization is needed per active block.

Inter-block plan. Used when the pair relation spans two or more blocks in a regular pattern. The backend groups logical pair partners by block identifier, acquires the required block set, executes the dense kernel over those slot addresses, and marks each modified block dirty. A practical implementation should acquire the full group before kernel execution so that the dense kernel sees stable addresses for the whole update.

Generic multi-block plan. Used for broader access patterns such as arbitrary multi-qubit kernels. The backend builds the set of required logical blocks from the index map induced by the gate, acquires all of them, and dispatches a kernel that reads and writes through local offsets into the acquired block views. The same authority and dirty rules remain unchanged.

6.3 Controlled Gates

Controlled single-qubit gates keep the same target-driven pair relation as their uncontrolled counterpart and simply filter the active pair set using the control-mask condition. They therefore reuse the same planning logic: determine the partner stride from the target qubit, decide whether the access is intra-block or inter-block, acquire the required blocks, and apply the dense kernel only to logical pairs that satisfy the control predicate.

6.4 Two-Qubit and Larger Kernels

A two-qubit gate affects amplitude groups indexed by two target bits instead of one. The backend does not need a separate storage model for this case. It needs only two planning steps:

1. enumerate the logical indices touched by one kernel application pattern;
2. map those indices through $(j(i), o(i))$ to obtain the required block set and local offsets.

If all mapped indices fall in one logical block, the operation becomes a local block kernel. Otherwise it becomes a multi-block kernel using the generic plan.

7 Compression Pipeline and Backing Store

The architecture is compatible with several block codecs, including floating-point compressors and reversible preprocessing schemes [1, 6, 8]. The backend should treat the codec as a pluggable stage behind one fixed block contract.

7.1 Per-Block Pipeline

For each block, the physical write path is:

$$\text{dense amplitudes} \rightarrow \text{optional reversible transform} \rightarrow \text{compression} \rightarrow \text{payload write.}$$

The read path is the inverse:

$$\text{payload read} \rightarrow \text{decompression} \rightarrow \text{inverse transform} \rightarrow \text{dense amplitudes.}$$

The simulator never observes this pipeline. It requests block views; the backend decides how those views are reconstructed.

7.2 Backing Store Layout

An implementation can store compressed payloads in memory, on local disk, or in another backing medium. The architecture only requires that each block descriptor identifies where its current payload is stored and how long it is. Variable compressed sizes are expected, so the descriptor must support payload relocation after recompression. A practical storage manager can satisfy this requirement with either:

- an append-only payload region plus updated offsets, or
- a reusable free-space map for overwritten payload regions.

The architectural requirement is simple: the descriptor for block j must always point to the unique current payload of that block whenever the block is not dirty in cache.

8 Synchronization and State Export

At any point during execution, dirty cached blocks may contain newer data than compressed storage. The backend therefore needs explicit synchronization points where compressed storage is brought back into agreement with the logical state.

8.1 Synchronization Events

An implementation should synchronize at least on the following events.

- Before exporting the full state vector.
- Before serializing the compressed state as a stable checkpoint.
- Before releasing backend resources at the end of simulation.
- Before any operation that requires the compressed store itself to be globally current.

8.2 Correctness Statement

Proposition 2. If the index invariant, authority invariant, cache-slot invariant, and flush invariant hold, then the compressed backend is logically equivalent to a dense state-vector backend.

Reason. Every logical amplitude belongs to one unique block and local offset. Every kernel update is applied to the dense materialization of the unique block set that contains those logical amplitudes. After mutation, the updated dense copy is authoritative until writeback. A synchronization flush restores compressed storage to the same logical values. Therefore every logical read, gate result, and exported state matches the result of a dense implementation.

The proposition is enough for implementation: correctness depends on obeying the acquisition, dirty, flush, and eviction rules, not on any special property of the codec beyond exact block reconstruction.

9 Implementation Guidance and Design Choices

The architecture leaves room for engineering choices, but some choices are more important than others because they change runtime behavior directly.

Two implementation recommendations follow directly from the architecture.

First, cache and block metadata should be updated in one place only, ideally in the acquisition and flush routines. Centralizing those transitions keeps the authority rule simple and makes debugging feasible.

Second, gate planning should be separated from gate arithmetic. The planner computes the block set and local offsets; the dense kernel only consumes dense pointers and local index lists. This separation allows the simulator to reuse existing dense kernels while replacing only the storage and scheduling layer.

Table 3. Primary design parameters and their implementation effect.

Parameter	Implementation role	Practical effect
Block size B	Defines partition granularity.	Larger blocks often compress better and favor local execution for more gates; smaller blocks reduce miss cost and metadata locality.
Cache capacity C	Bounds dense residency.	Small caches increase decompression frequency; large caches approach dense storage behavior.
Replacement policy	Chooses the eviction victim.	LRU is simple and exploits temporal locality in repeated block use.
Codec choice	Defines block compression time and ratio.	Fast codecs reduce latency; stronger codecs may reduce the footprint further.
Writeback timing	Chooses when dirty blocks are recompressed.	Delayed writeback improves reuse; frequent flushes simplify memory turnover.
Backing-store manager	Tracks payload locations.	Needed because block payload sizes vary after recompression.

10 Related Work

Dense state-vector simulators remain the reference model for exact quantum circuit execution and expose the memory bottleneck that motivates the present design [5, 10, 4, 3, 2]. Alternative representations such as tensor networks and decision diagrams reduce memory by changing the representation itself [11, 7, 12]. The architecture developed here stays within the conventional state-vector model and instead reorganizes how that model is stored and recovered.

Compression-assisted simulation has already shown that amplitude data can sometimes be stored more compactly and that compression ratio depends strongly on the circuit family and on the numerical structure of the simulated state [13–15]. The present work addresses a different layer of the problem: not whether compression can help, but how a simulator backend must be structured so that compressed storage can safely serve as the primary execution-time representation. The key architectural elements are contiguous block partitioning, a unique authority rule per block, bounded dense materialization, explicit dirty writeback, and gate planning over the induced block layout.

11 Conclusion

This paper presented a compressed-block backend architecture for full-state quantum circuit simulation at the level of direct implementation. The design

keeps a logical dense state-vector interface for the simulator while storing the physical state as independent compressed blocks plus a bounded cache of dense materializations. Its core mechanisms are the logical-to-physical index map, the block descriptor, the three-state cache machine, the deterministic acquisition protocol, and the gate planner that converts logical gate semantics into block-local execution plans.

The value of the architecture is practical clarity. A developer implementing compressed execution for a Schrödinger-style simulator needs to decide how blocks are represented, when dense views are created, how stale compressed payloads are avoided, and how gate access traverses the block layout. Those decisions are made explicit here as backend rules and structures, so the compressed representation becomes a controllable execution substrate rather than an external storage add-on.

Acknowledgments. This work is part of an undergraduate research project at Universidad Industrial de Santander on memory management for software quantum simulators.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Burtscher, M., Ratanaworabhan, P.: FPC: A high-speed compressor for double-precision floating-point data. IEEE Transactions on Computers **58**(1), 18–31 (2009). https://doi.org/10.1109/TC.2008.131
2. Díaz, G., Steffenel, L., Barrios, C., Couturier, J.: Memory management strategies for software quantum simulators. Quantum Reports **7**(3) (2025). https://doi.org/10.3390/quantum7030041
3. Díaz, G., Steffenel, L.A., Barrios, C.J., Couturier, J.F.: How to build a software quantum simulator (2024). https://doi.org/10.20944/preprints202409.1497.v1, `https://www.preprints.org/manuscript/202409.1497/v1`
4. Häner, T., Steiger, D.S.: 0.5 petabyte simulation of a 45-qubit quantum circuit. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (2017). https://doi.org/10.1145/3126908.3126947
5. Jones, T., Brown, A., Bush, I., Benjamin, S.C.: QuEST and high performance simulation of quantum computers. Scientific Reports **9**, 10736 (2019). https://doi.org/10.1038/s41598-019-47174-9
6. Lindstrom, P., Isenburg, M.: Fast and efficient compression of floating-point data. IEEE Transactions on Visualization and Computer Graphics **12**(5), 1245–1250 (2006). https://doi.org/10.1109/TVCG.2006.143
7. Markov, I.L., Shi, Y.: Simulating quantum computation by contracting tensor networks. SIAM Journal on Computing **38**(3), 963–981 (2008). https://doi.org/10.1137/050644756
8. Masui, K., Amiri, M., Connor, L., Deng, M., Fandino, M., Höfer, C., Halpern, M., Hanna, D., Hincks, A.D., Hinshaw, G., Parra, J.M., Newburgh, L.B., Shaw, J.R., Vanderlinde, K.: A compression scheme for radio data in high performance computing. Astronomy and Computing **12**, 181–190 (2015). https://doi.org/10.1016/j.ascom.2015.07.002

9. Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information. Cambridge University Press, 10th anniversary edition edn. (2010)
10. Steiger, D.S., Häner, T., Troyer, M.: ProjectQ: An open source software framework for quantum computing. Quantum **2**, 49 (2018). https://doi.org/10.22331/q-2018-01-31-49
11. Vidal, G.: Efficient classical simulation of slightly entangled quantum computations. Physical Review Letters **91**(14), 147902 (2003). https://doi.org/10.1103/PhysRevLett.91.147902
12. Vinkhuijzen, L., Coopmans, T., Elkouss, D., Dunjko, V., Laarman, A.: LIMDD: A decision diagram for simulation of quantum computing including stabilizer states. Quantum **7**, 1108 (2023). https://doi.org/10.22331/q-2023-09-11-1108
13. Wu, X.C., Di, S., Cappello, F., Finkel, H., Alexeev, Y., Chong, F.T.: Amplitude-aware lossy compression for quantum circuit simulation. In: Proceedings of the 4th International Workshop on Data Reduction for Big Scientific Data. IEEE (2018), `https://sc18.supercomputing.org/proceedings/workshops/workshop_files/ws_drbsd112s1-file1.pdf`
14. Wu, X.C., Di, S., Dasgupta, E.M., Cappello, F., Finkel, H., Alexeev, Y., Chong, F.T.: Full-state quantum circuit simulation by using data compression. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE/ACM (2019). https://doi.org/10.1145/3295500.3356155
15. Zhang, B., Fang, B., Ye, F., Guo, L., Tallent, N., Tao, D.: BMQSim: Overcoming memory constraints in quantum circuit simulation with a high-fidelity compression framework. In: Proceedings of the 39th ACM International Conference on Supercomputing (2025). https://doi.org/10.1145/3721145.3725747